CRENMA Discovery Agent — Registration & Uninstallation Guide

This document walks you through **(A) building the installer**, **(B) registering/installing the Discovery Agent on a customer machine**, and **(C) uninstalling the agent safely** using the two scripts you provided.

Part 0 — Overview

Component	Purpose
`build_installer.sh` (your first script)	**Build machine only.** Bundles the local `discovery_agent` source into a single self-contained `crenma-discovery-agent-installer.sh`.
`crenma-discovery-agent-installer.sh` (generated)	**Customer machine.** Installs the agent, registers it with CRENMA SaaS, and sets up the systemd service.
`uninstall.sh` (your second script)	**Customer machine.** Deactivates the agent in CRENMA SaaS, then removes all local files.

Key facts:

- The customer machine needs **only**: Linux, root/sudo, systemd, and network access to CRENMA.
- The customer does **NOT** need Python, pip, virtualenv, PyInstaller, Git, or source code.
- Registration happens **at install time** via the installer (activation binary).
- Deactivation happens **at uninstall time** via a signed API request to CRENMA.

Part A — Build the Installer (Build Machine)

A.1 Prerequisites on the build machine

- Linux or WSL
- Python 3, `pip` (internet access for build deps)
- `tar`, `gzip`, `base64`, `sha256sum`, `file`, `mktemp`, `awk`, `sed`, `grep`, `find`

A.2 Expected project layout

```
<PROJECT_DIR>/
├── build_installer.sh              # your first script
├── installer.sh                    # installer template (must contain
marker)
├── requirements.txt
└── discovery_agent/
    ├── __main__.py
    ├── activate.py
    ├── database/
    │   └── database.py
    └── renewal/
```

The installer template **must** contain the marker line exactly once:

```
__CRENMA_DISCOVERY_PAYLOAD__
```

Place this marker **after** all shell logic in `installer.sh` (and ideally after an `exit 0` line), because the build script appends the base64 payload right after it.

A.3 Build steps

1. **Place the scripts in the project root** (same folder as `discovery_agent/`).
2. **(Optional) Set the default server URL** used at install time:

```
export DEFAULT_SERVER_URL="https://qa.crenma.com"
```

The build script will replace @DEFAULT_SERVER_URL@ in your installer template with this value.

3. **(Optional) Control build cleanup:**

```
export KEEP_BUILD_ENV=1          # keep the build venv
export KEEP_BUILD_ARTIFACTS=1    # keep build/ and .pyinstaller-build/
export KEEP_PAYLOAD=1            # keep payload dir/tarball
```

4. **Run the builder:**

```
chmod +x build_installer.sh
./build_installer.sh
```

A.4 What the builder does (in order)

1. Detects build OS + architecture (`x86_64` or `aarch64`).
2. Validates the local source tree, `requirements.txt`, and installer template.
3. Verifies the installer template contains **exactly one** payload marker.
4. Generates PyInstaller specs for:
    a. `crenma-discovery-agent` (from `discovery_agent/__main__.py`)
    b. `crenma-discovery-activate` (from `discovery_agent/activate.py`)
5. Inspects `discovery_agent/renewal/` for external `certbot`/subprocess usage and warns if found.
6. Creates a fresh build venv, installs PyInstaller + app requirements.
7. Verifies source imports (`discovery_agent`, `__main__`, `activate`, database, SQLAlchemy).
8. Builds both binaries.
9. Normalizes binary names into `dist/`.
10. Verifies binaries exist, are executable, non-empty, and match the build architecture.
11. Creates `discovery-payload.tar.gz` with both binaries.
12. Computes SHA256 checksums.
13. Prepares the installer template (injects server URL).
14. Embeds the base64 payload into the final installer after the marker.
15. Validates shell syntax and extracts the payload back out to verify SHA256 matches.
16. Cleans intermediates (unless `KEEP_*` flags are set).

17. Prints a summary.

A.5 Build output

```
crenma-discovery-agent-installer.sh
```

This single file is what you ship to the customer.

Verify it before shipping:

```
bash -n crenma-discovery-agent-installer.sh
ls -lh crenma-discovery-agent-installer.sh
```

Part B — Register / Install the Agent (Customer Machine)

B.1 Requirements

- Linux with **systemd**
- **root** (or sudo)
- Network access to the CRENMA server (default https://qa.crenma.com)
- Supported CPU: x86_64 or aarch64

B.2 Copy the installer to the customer machine

```
scp crenma-discovery-agent-installer.sh user@customer-host:/tmp/
```

B.3 Run the installer as root

```
sudo bash /tmp/crenma-discovery-agent-installer.sh
```

B.4 What happens during installation

The installer template (your `installer.sh`) is executed with the embedded payload. Typically it will:

1. **Extract the payload** — decode the base64 blob appended after the marker and untar it into a temp dir.
2. **Verify checksums** of `crenma-discovery-agent` and `crenma-discovery-activate`.
3. **Install binaries** to `/discovery_agent/` (and possibly `/usr/local/bin/`).
4. **Run the activation step** — execute `crenma-discovery-activate` to register the agent with CRENMA SaaS. This is where:
    a. The agent is enrolled against the server (`DEFAULT_SERVER_URL`).
    b. Cryptographic keys are generated and stored under `/etc/crenma-discovery-agent/keys/`:
        i. `agent_signing_private_key.pem`
        ii. `agent_signing_public_key.pem`
    c. An identity file is written to `/etc/crenma-discovery-agent/agent_identity.json` containing `org_id` and `agent_id`.
    d. Installation secrets may be written to `/discovery_agent/config/secrets.json`.
5. **Install the systemd unit** at `/etc/systemd/system/crenma-discovery-agent.service`.
6. **Enable and start the service**:

```
systemctl daemon-reload
systemctl enable crenma-discovery-agent
systemctl start crenma-discovery-agent
```

B.5 Verify the installation

```
# Service status
systemctl status crenma-discovery-agent

# Recent logs
journalctl -u crenma-discovery-agent -n 100 --no-pager

# Installed files
ls -l /discovery_agent/
ls -l /etc/crenma-discovery-agent/keys/
cat /etc/crenma-discovery-agent/agent_identity.json
```

You should see:

- A running service
- A non-empty `org_id` and `agent_id`
- Both signing keys present

B.6 Registration flow summary

```
Installer  ⟶  extract payload
           ⟶  verify SHA256
           ⟶  install binaries
           ⟶  run activate binary  ⟶  CRENMA SaaS
                                        (enroll agent, receive
org_id/agent_id)
           ⟶  write keys + identity file
           ⟶  install + start systemd service
```

Part C — Uninstall the Agent (Customer Machine)

Use the `uninstall.sh` script. It performs a **signed remote deactivation first**, then removes local files.

C.1 Requirements

- root
- `bash`, `curl`, `openssl`, `base64`, `date`
- Optional: `python3` (used only as a fallback to read `org_id`/`agent_id` from JSON)

C.2 How identity is recovered

The script tries, in order:

1. Environment variables `CRENMA_ORG_ID` / `CRENMA_AGENT_ID`
2. `/etc/crenma-discovery-agent/agent_identity.json`
3. `/discovery_agent/config/secrets.json`

If none yield both values, the script aborts.

If your install dir was already wiped, you can still uninstall by exporting:

```
export CRENMA_ORG_ID="<org-id>"
export CRENMA_AGENT_ID="<agent-id>"
```

C.3 Uninstall steps

1. **Copy the uninstaller to the customer machine:**

```
scp uninstall.sh user@customer-host:/tmp/
```

2. **Run it as root:**

```
sudo bash /tmp/uninstall.sh
```

Flags:

   a. `--yes` — skip confirmation prompt
   b. `--force` — proceed with local cleanup even if SaaS deactivation fails (⚠ not recommended)
   c. `--help` — usage

3. **Confirm when prompted** (unless `--yes`).

C.4 What the uninstaller does

1. **Recovers identity** (`org_id`, `agent_id`).
2. **Validates signing key** at `/etc/crenma-discovery-agent/keys/agent_signing_private_key.pem`.
3. **Reconstructs the public key** if missing (derived from the private key — never generates a new pair).
4. **Verifies the key pair matches** by comparing SHA256 fingerprints of the DER-encoded public keys.
5. **Derives the raw Ed25519 public key** (last 32 bytes of SubjectPublicKeyInfo DER) — used by the SaaS to look up the agent.
6. **Builds a signed deactivation request:**

   a. Canonical JSON: `{"agent_id":"...","nonce":"...","org_id":"...","timestamp":N}` (sorted keys, compact, no `reason`)
   b. Ed25519 signature via `openssl pkeyutl -sign -rawin`
   c. Base64-encoded signature
   d. Request body: `{"reason":"Agent uninstallation"}`
7. **POSTs to** `POST {SERVER_URL}/v1/db/agent/deactivate` with headers:
   a. `Content-Type: application/json`
   b. `X-Org-Id`, `X-Agent-Id`, `X-Timestamp`, `X-Nonce`, `X-Signature`
8. **Interprets the response:**
   a. HTTP 200 + `"status":"deactivated"` → success
   b. HTTP 200 + `"already_deactivated":true` → success
   c. Otherwise → failure
   d. On curl failure or non-200: aborts unless `--force`
9. **On confirmed deactivation, performs local cleanup:**
   a. Stops & disables `crenma-discovery-agent.service`
   b. Backs up keys + install dir + journal to `/var/backups/crenma-discovery-agent/uninstall_<timestamp>/` (or to `/discovery_agent/backup/...` if that dir still exists)
   c. Removes `/etc/systemd/system/crenma-discovery-agent.service`
   d. Removes `/discovery_agent/`
   e. Removes `/etc/crenma-discovery-agent/`
10. **Prints a verification summary.**

C.5 Overriding the server URL

```
export CRENMA_SERVER_URL="https://prod.crenma.com"
sudo -E bash /tmp/uninstall.sh
```

C.6 Verify the uninstallation

```
# Service should be gone
systemctl status crenma-discovery-agent   # expect: not found

# Files should be gone
ls /discovery_agent/                       # expect: No such file or
directory
```

```
ls /etc/crenma-discovery-agent/            # expect: No such file or
directory

# Backup retained
ls /var/backups/crenma-discovery-agent/
```

Part D — Troubleshooting

Symptom	Cause / Fix
Build fails: "Expected exactly one payload marker"	`installer.sh` must contain the marker `__CRENMA_DISCOVERY_PAYLOAD__` exactly once.
Build fails: "Could not locate Discovery Agent PyInstaller output"	Check `.build-logs/agent.log` (kept on failure).
Build fails: import errors	Ensure `requirements.txt` is complete and `discovery_agent/__init__.py` exists.
Uninstaller: "Registered agent identity could not be recovered"	Export `CRENMA_ORG_ID` and `CRENMA_AGENT_ID`, or restore the identity file.
Uninstaller: "Signing private key is missing"	Use `--force` to skip SaaS deactivation (⚠ leaves the agent active in SaaS).
Uninstaller: "Signing private/public key pair does not match"	The public key file was tampered with. Delete the public key file — the script will re-derive it from the private key.
Uninstaller: "SaaS deactivation failed with HTTP 4xx"	Signature mismatch, expired timestamp, or the agent was already removed server-side. Check the printed response body.

Uninstaller: curl exit code ≠ 0	Network/firewall issue. Verify `CRENMA_SERVER_URL` is reachable.
Uninstaller: HTTP 200 but no `deactivated` status	The SaaS returned a different payload. Inspect the printed response body; do not use `--force` unless you accept remote revocation being unconfirmed.

Part E — Quick Reference

Build (once, on build machine)

```
export DEFAULT_SERVER_URL="https://qa.crenma.com"
./build_installer.sh
# → crenma-discovery-agent-installer.sh
```

Install (per customer machine)

```
sudo bash crenma-discovery-agent-installer.sh
systemctl status crenma-discovery-agent
```

Uninstall (per customer machine)

```
sudo bash uninstall.sh                # interactive
sudo bash uninstall.sh --yes          # non-interactive
sudo bash uninstall.sh --force        # local cleanup even if SaaS fails (⚠)
```

Key paths

Path	Purpose
`/discovery_agent/`	Agent installation dir

/etc/crenma-discovery-agent/keys/agent_signing_private_key.pem	Ed25519 signing private key
/etc/crenma-discovery-agent/keys/agent_signing_public_key.pem	Ed25519 signing public key
/etc/crenma-discovery-agent/agent_identity.json	org_id + agent_id
/etc/systemd/system/crenma-discovery-agent.service	systemd unit
/var/backups/crenma-discovery-agent/uninstall_<ts>/	Uninstall backup

Part F — Complete Example: End-to-End Registration and Uninstall

This section shows a **full copy-paste-ready session** covering the entire lifecycle.

F.1 On the BUILD machine

```
# 1. Enter the project directory
cd /path/to/crenma-discovery-agent-project

# 2. (Optional) Set the default CRENMA server URL used during install
export DEFAULT_SERVER_URL="https://qa.crenma.com"

# 3. Make the builder executable and run it
chmod +x build_installer.sh
./build_installer.sh

# 4. Confirm the single-file installer was produced
ls -lh crenma-discovery-agent-installer.sh
bash -n crenma-discovery-agent-installer.sh   # syntax check
```

Expected final output:

```
============================================================
 Crenma Discovery Agent Installer Build Complete
============================================================

Build architecture:
  x86_64

Source:
  /path/to/project/discovery_agent

Default server URL:
  https://qa.crenma.com

Discovery Agent SHA256:
  <hex>

Activation SHA256:
  <hex>

Final installer:
  /path/to/project/crenma-discovery-agent-installer.sh

Installer size:
  <size>

Customer delivery:
  /path/to/project/crenma-discovery-agent-installer.sh
...
```

F.2 On the CUSTOMER machine — install / register

```
# 1. Copy the installer
scp crenma-discovery-agent-installer.sh user@customer-host:/tmp/

# 2. SSH into the customer host
ssh user@customer-host

# 3. Run the installer as root
```

```
sudo bash /tmp/crenma-discovery-agent-installer.sh

# 4. Verify the service is running
sudo systemctl status crenma-discovery-agent

# 5. Verify registration artifacts
sudo cat /etc/crenma-discovery-agent/agent_identity.json
sudo ls -l /etc/crenma-discovery-agent/keys/
sudo ls -l /discovery_agent/
```

Expected: a running `crenma-discovery-agent.service`, a JSON identity file with `org_id` and `agent_id`, and both `agent_signing_private_key.pem` / `agent_signing_public_key.pem` present.

F.3 On the CUSTOMER machine — uninstall / deactivate

```
# 1. Copy the uninstaller
scp uninstall.sh user@customer-host:/tmp/

# 2. SSH into the customer host
ssh user@customer-host

# 3. (Optional) Override the server URL if not using the default
# export CRENMA_SERVER_URL="https://prod.crenma.com"

# 4. Run the uninstaller as root
sudo -E bash /tmp/uninstall.sh

# 5. When prompted, confirm
# Continue? [y/N]: y

# 6. Verify removal
sudo systemctl status crenma-discovery-agent     # expect: not found
sudo ls /discovery_agent/                         # expect: No such
file or directory
sudo ls /etc/crenma-discovery-agent/              # expect: No such
file or directory
sudo ls /var/backups/crenma-discovery-agent/      # expect: a backup
```

```
directory with timestamp
```

Expected final output:

```
============================================================
 CRENMA Discovery Agent Uninstallation Complete
============================================================
```

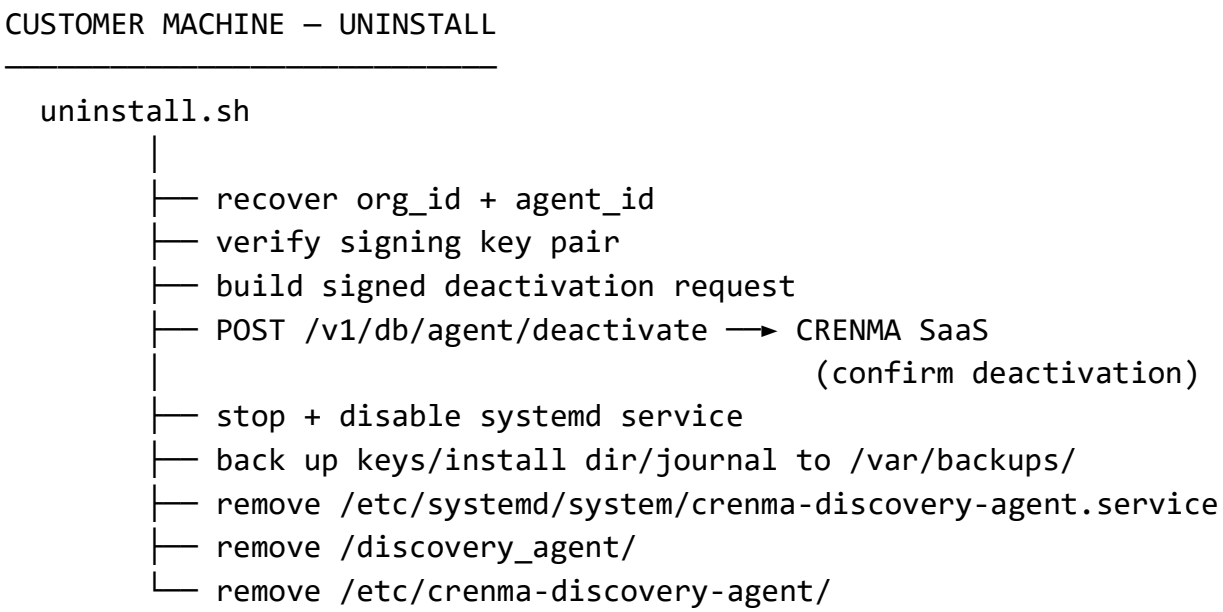
F.4 Full lifecycle diagram

```
BUILD MACHINE
─────────────

  build_installer.sh
        │
        ├── validate source + template
        ├── PyInstaller → crenma-discovery-agent
        ├── PyInstaller → crenma-discovery-activate
        ├── tar + base64 payload
        └── embed into installer.sh
                    │
                    ▼
        crenma-discovery-agent-installer.sh   ──► ship to customer


CUSTOMER MACHINE — INSTALL
──────────────────────────

  crenma-discovery-agent-installer.sh
        │
        ├── extract payload
        ├── verify SHA256
        ├── install binaries to /discovery_agent/
        ├── run crenma-discovery-activate ──► CRENMA SaaS
        │                                     (enroll, get
org_id/agent_id)
        ├── write keys to /etc/crenma-discovery-agent/keys/
        ├── write identity to /etc/crenma-discovery-
agent/agent_identity.json
        └── install + enable + start systemd service
```



F.5 Summary checklist

Build machine:

- `discovery_agent/`, `requirements.txt`, `installer.sh` all present
- `installer.sh` contains `__CRENMA_DISCOVERY_PAYLOAD__` exactly once
- `DEFAULT_SERVER_URL` exported (if overriding default)
- `./build_installer.sh` completes with SUCCESS
- `crenma-discovery-agent-installer.sh` produced and syntax-valid

Customer machine — install:

- Installer copied and executed as root
- `systemctl status crenma-discovery-agent` → active (running)
- `/etc/crenma-discovery-agent/agent_identity.json` contains `org_id` and `agent_id`
- Signing keys present under `/etc/crenma-discovery-agent/keys/`

Customer machine — uninstall:

- Uninstaller copied and executed as root

- SaaS response confirms `deactivated` (or `already_deactivated`)
- `/discovery_agent/`, `/etc/crenma-discovery-agent/`, and the systemd unit are gone
- Backup exists at `/var/backups/crenma-discovery-agent/uninstall_<ts>/`